Research Report: Local LLM Integration with ComfyUI on Dual RTX 3090 Hardware

DATE: 2026-03-22
OBJECTIVE: To analyze local Large Language Model (LLM) options suitable for a hardware configuration of 2x NVIDIA RTX 3090 GPUs (48GB total VRAM) and 128GB RAM, focusing on integration with ComfyUI for creative reasoning and workflow generation.

Executive Summary

This report provides a comprehensive analysis of deploying local Large Language Models (LLMs) on a system equipped with two NVIDIA RTX 3090 GPUs, offering a combined 48GB of VRAM. The findings indicate that this hardware configuration is highly capable of running sophisticated, large-scale models for creative and instructional tasks within a ComfyUI environment.

Key findings are as follows:
* **Model Capability:** The 48GB VRAM capacity is sufficient to run 70-billion-parameter class models, such as Llama 3 70B and Qwen2 72B, using 4-bit quantization. These models typically require 35-45GB of VRAM, leaving a comfortable buffer for context length and operational overhead.
* **Inference Frameworks:** For multi-GPU performance, **vLLM** emerges as the superior choice due to its advanced memory management (PagedAttention), high throughput, and robust support for tensor and pipeline parallelism. While alternatives like Ollama offer ease of use, they lack the performance and multi-GPU optimization necessary to fully leverage this hardware.
* **Performance:** Benchmarks show that a dual RTX 3090 setup can achieve respectable inference speeds, such as approximately 16 tokens per second for a Llama 3 70B model at 4-bit quantization. Performance is significantly enhanced by using an NVLink bridge, which can increase inference speed by up to 50% in a dual-GPU configuration.
* **ComfyUI Integration:** Sharing GPU resources between an LLM and ComfyUI is feasible. Strategies include dedicating one GPU to the LLM and the other to ComfyUI, or using advanced custom nodes like **ComfyUI-MultiGPU** to intelligently offload model components to system RAM or a secondary GPU. This allows for concurrent operation without VRAM exhaustion.
* **Advanced Capabilities:** Modern models like Llama 3 and Qwen support function calling and tool use, enabling them to interact with external APIs and data sources. This transforms them from simple text generators into powerful agents capable of executing complex workflows, a capability that can be integrated into ComfyUI via custom nodes that connect to LLM API servers.

In conclusion, a dual RTX 3090 system provides a powerful and cost-effective platform for local AI development, capable of running near state-of-the-art open-source models. Success hinges on selecting the right inference framework (vLLM), employing 4-bit quantization, and utilizing specific ComfyUI extensions for resource management.

Hardware Capability and VRAM Analysis

A system with two NVIDIA RTX 3090 GPUs, each with 24GB of VRAM, provides a total of 48GB of VRAM. This memory capacity is the primary determinant of which LLMs can be run effectively.

VRAM Requirements and Quantization

The memory footprint of an LLM is dictated by its parameter count and the precision of its weights. Quantization is a critical technique that reduces this footprint by lowering the precision (e.g., from 16-bit floating point to 4-bit integer) with a minimal impact on performance.

- **FP16 (16-bit):** A 70B parameter model in its native 16-bit format would require approximately 140GB of VRAM, making it impossible to run on this hardware.
- **INT8 (8-bit):** Quantizing to 8-bit cuts the requirement in half. A 70B model would need ~70GB, still exceeding the available 48GB.
- **INT4 (4-bit):** 4-bit quantization reduces the VRAM requirement by approximately 75% compared to FP16. This is the key to running large models on consumer hardware.

With 48GB of VRAM, the following model classes are well within reach:

- **Llama 3 70B:** Using 4-bit quantization (e.g., Q4_K_M GGUF or AWQ), this model requires approximately **42-45GB of VRAM**. This leaves a small but sufficient buffer for the KV cache (which stores context) and framework overhead.
- **Qwen2 72B:** The 4-bit quantized version of this model requires approximately **35-36GB of VRAM**, fitting comfortably within the dual-3090 setup.
- **Mixtral 8x7B:** This Mixture-of-Experts (MoE) model has ~47B total parameters. At 4-bit quantization, it requires **26-28GB of VRAM**, easily fitting onto the hardware and even leaving one GPU almost entirely free for other tasks.

Models larger than ~72B, such as Mistral Large 2 (123B), are generally not feasible as they require over 70GB of VRAM even with 4-bit quantization.

Recommended LLM Models for Creative Tasks

For creative reasoning, workflow generation, and instruction following, several open-source models stand out.

- **Llama 3 Series:** Developed by Meta, the Llama 3 models, particularly the 70B variant, are highly regarded for their strong reasoning, instruction-following capabilities, and overall performance, often competing with proprietary models.
- **Qwen 2 Series:** Developed by Alibaba, the Qwen2 models, including the 72B version, demonstrate excellent multilingual support and strong performance in coding and mathematics. The family also includes vision-language (VL) variants that can interpret images.
- **Mistral / Mixtral:** Mistral AI's models are known for their efficiency and strong performance. While Mistral Large is too big, the Mixtral 8x7B model is an excellent choice, offering high performance with a lower VRAM footprint than dense 70B models.
- **DeepSeek Coder:** For tasks heavily focused on code generation and programming-related workflow automation, the DeepSeek Coder series is a specialized and powerful option.

For the specified hardware, **Llama 3 70B** and **Qwen2 72B** at 4-bit quantization represent the best balance of cutting-edge capability and hardware feasibility.

Multi-GPU Inference Frameworks and Servers

To run a single LLM across two GPUs, a specialized inference server that supports model parallelism is required. This involves splitting the model's layers or operations across the available VRAM.

Framework	Multi-GPU Support	Performance	Ease of Use	Best For
vLLM	Excellent (Tensor & Pipeline Parallelism)	**Very High**. Uses PagedAttention for efficient memory management and high throughput.	Medium. Requires command-line setup but is well-documented.	Production and high-performance local serving on NVIDIA GPUs. The recommended choice for this hardware.
Text Generation Inference (TGI)	Good (Tensor Parallelism)	**High**. Optimized for Hugging Face models with dynamic batching.	Medium. Integrates well with the Hugging Face ecosystem.	Teams heavily invested in Hugging Face tooling.
Ollama	Limited / Sequential	**Low**. Primarily designed for ease of use on a single GPU. Processes requests sequentially, limiting throughput.	**Very Easy**. Single-command installation and model management.	Local development, prototyping, and single-user scenarios. Not ideal for maximizing multi-GPU performance.
LocalAI	Limited	Low-Medium.	Easy.	Multi-modal applications requiring an all-in-one solution for various AI tasks, not just LLM inference.
llama.cpp	Good	High. CPU and GPU support with various quantization formats (GGUF).	Medium.	Running GGUF models efficiently with good CPU/GPU flexibility.

Recommendation: vLLM is the most suitable framework for this use case. Its native support for tensor parallelism allows it to efficiently split a large model across the two RTX 3090s, maximizing performance.

Performance Benchmarks and Considerations

Performance, measured in tokens per second (t/s), depends on the model, quantization, and framework.

- **Dual RTX 3090 Performance:** Benchmarks using vLLM show that two RTX 3090s can achieve:
  - **~16.29 t/s** for Llama 3 70B (Q4_K_M).
  - **~47.15 t/s** for Llama 3 8B (FP16).
- **NVLink Bridge:** The RTX 3090 supports NVLink, a high-speed interconnect that allows the two GPUs to share memory much faster than via the standard PCIe bus. For a dual-GPU setup, using an NVLink bridge is highly recommended as it can provide a **performance increase of up to 50%** for LLM inference by reducing communication overhead.
- **Power and Cooling:** Two RTX 3090s under load can draw 700-900W. A robust power supply (1200W or higher) and a well-ventilated case are essential for stable operation.

While a single, more powerful GPU is simpler, the dual 3090 configuration provides the raw VRAM necessary for 70B models, which is not possible on any single consumer GPU (including the RTX 4090 with 24GB). The resulting inference speed of ~16 t/s is very usable for interactive tasks and workflow automation.

Integrating LLMs with ComfyUI

Running an LLM and ComfyUI simultaneously requires careful management of GPU resources to avoid VRAM conflicts.

Resource Sharing Strategies

1. **Dedicated GPUs:** The simplest approach is to assign the LLM server to one GPU and ComfyUI to the other. This can be done using the `CUDA_VISIBLE_DEVICES` environment variable when launching each application. For example, run the LLM on GPU 0 and ComfyUI on GPU 1. This isolates the workloads completely.
2. **Dynamic Offloading with Custom Nodes:** A more flexible approach involves using ComfyUI extensions designed for advanced VRAM management. The **ComfyUI-MultiGPU** custom node is particularly powerful. It allows users to:
   - Offload parts of a model (like ComfyUI's UNet or CLIP) to a secondary GPU or system RAM.
   - Free up VRAM on the primary GPU, allowing an LLM to run alongside ComfyUI on the same device.
   - Precisely allocate model shards across devices, for example, loading 2.5GB of a model onto `cuda:0` and the rest onto the CPU.

LLM Integration Nodes for ComfyUI

Several custom nodes facilitate direct communication between ComfyUI and an LLM:

- **ComfyUI-OpenAI-Compat-LLM-Node:** This node connects to any OpenAI-compatible API endpoint. You can run a local vLLM or Ollama server, which provides such an endpoint, and use this node to send prompts (including images for multimodal models) to your local LLM and receive generated text back into your ComfyUI workflow. This is ideal for using the LLM to generate creative prompts for image generation.
- **ComfyUI_LocalLLMNodes:** This node pack is designed to run LLMs (GGUF or Hugging Face format) directly inside ComfyUI for prompt generation. It can take a simple description and generate a detailed, complex prompt suitable for models like Stable Diffusion.

- **ComfyUI_VLM_nodes:** This collection focuses on Vision Language Models (VLMs), enabling image-to-text, image analysis, and other multimodal interactions directly within the workflow.

The recommended workflow is to run the LLM using a dedicated server like vLLM (on one or both GPUs) and connect to it from ComfyUI using the `ComfyUI-OpenAI-Compat-LLM-Node`. This decouples the applications and provides stable, high-performance LLM access.

Advanced Capabilities: Tool Use and Function Calling

Modern LLMs can be transformed into intelligent agents through **function calling** or **tool use**. This allows the model to interact with external systems by generating structured JSON output that calls a predefined function or API.

- **How it Works:** A developer provides the LLM with a list of available tools and their descriptions (e.g., a `get_weather(location)` function). When a user asks, "What's the weather in San Francisco?", the LLM recognizes the need for the tool and outputs a request to call `get_weather` with the parameter `location="San Francisco"`. The application code executes this function, gets the real-world data, and feeds it back to the LLM to formulate a final answer.
- **Frameworks and Models:**
  - Models like **Qwen** and recent versions of **Llama 3** have been fine-tuned for function calling.
  - Frameworks like **Qwen-Agent** provide a Python-based implementation for this.
  - For open-source models, libraries like **Instructor** and **Outlines** help constrain the LLM's output to a valid JSON schema, making function calls more reliable.
- **Use in ComfyUI:** This capability can be used to create highly autonomous workflows. For example, an LLM agent within ComfyUI could be instructed to "generate an image of a futuristic city." The agent could first use a tool to search the web for "futuristic city concepts," use that information to generate a detailed prompt, pass the prompt to Stable Diffusion, and then use another tool to post the final image to a social media platform.

By running a local LLM server that supports function calling, these agent-like behaviors can be built directly into custom ComfyUI workflows.

References

1. Self-hosting Llama 3.1 70b or any 70b LLM affordably - Medium (https://abhinand05.medium.com/self-hosting-llama-3-1-70b-or-any-70b-llm-affordably-2bd323d72f8d)
2. Can You Run This LLM? VRAM Calculator (Nvidia GPU and Apple Silicon) - apxml.com (https://apxml.com/tools/vram-calculator)
3. GPU System Requirements for Qwen Models - apxml.com (https://apxml.com/posts/gpu-system-requirements-qwen-models)
4. Qwen2.5-72B - apxml.com (https://apxml.com/models/qwen2-5-72b)
5. Guide to GPU Requirements for Running AI Models - Bacloud (https://www.bacloud.com/en/blog/163/guide-to-gpu-requirements-for-running-ai-models.html)
6. Function Calling with Open-Source LLMs - BentoML (https://www.bentoml.com/blog/function-calling-with-open-source-llms)
7. Qwen2.5-VL-72B: VRAM, Performance, and Deployment Guide - Novita AI Blogs (https://blogs.novita.ai/qwen2-5-vl-72b-vram/)

8. LLM Serving: Ollama vs vLLM vs TGI | Guides | Clore.ai - Clore.ai (https://docs.clore.ai/guides/comparisons/llm-serving-comparison)
9. Multi-node & Multi-GPU inference with vLLM - MeluXina User Documentation - MeluXina User Documentation (https://docs.lxp.lu/howto/llama3-vllm/)
10. Distributed Inference and Serving — vLLM - vLLM (https://docs.vllm.ai/en/v0.8.0/serving/distributed_serving.html)
11. Parallelism and Scaling - vLLM - vLLM (https://docs.vllm.ai/en/stable/serving/parallelism_scaling/)
12. Deep Learning VRAM Calculator - GPU Memory Requirements for LLMs & AI | DataZier - datazier.com (https://www.datazier.com/free-tools/vram-calculator/)
13. ComfyUI-Workflow/awesome-comfyui - GitHub (https://github.com/ComfyUI-Workflow/awesome-comfyui)
14. Big-Idea-Technology/ComfyUI_LLM_Node - GitHub (https://github.com/Big-Idea-Technology/ComfyUI_LLM_Node)
15. gokayfem/ComfyUI_VLM_nodes - GitHub (https://github.com/gokayfem/ComfyUI_VLM_nodes)
16. kmlbdh/ComfyUI_LocalLLMNodes - GitHub (https://github.com/kmlbdh/ComfyUI_LocalLLMNodes)
17. lxe/ComfyUI-OpenAI-Compat-LLM-Node - GitHub (https://github.com/lxe/ComfyUI-OpenAI-Compat-LLM-Node)
18. pollockjj/ComfyUI-MultiGPU - GitHub (https://github.com/pollockjj/ComfyUI-MultiGPU)
19. vllm-project/vllm/issues/7228 - GitHub (https://github.com/vllm-project/vllm/issues/7228)
20. XiongjieDai/GPU-Benchmarks-on-LLM-Inference - GitHub (https://github.com/XiongjieDai/GPU-Benchmarks-on-LLM-Inference)
21. vLLM vs TGI vs TensorRT-LLM vs Ollama | Hivenet - Hivenet (https://compute.hivenet.com/post/vllm-vs-tgi-vs-tensorrt-llm-vs-ollama)
22. Multi-GPU LLM Serving: A Practical Guide | Hivenet - Hivenet (https://compute.hivenet.com/post/multi-gpu-llm-serving)
23. vLLM Performance Benchmarks (4x RTX 3090) - himeshp.blogspot.com (http://himeshp.blogspot.com/2025/03/vllm-performance-benchmarks-4x-rtx-3090.html)
24. Discussion on Llama-3.1-Nemotron-70B-Instruct-HF - Hugging Face (https://huggingface.co/nvidia/Llama-3.1-Nemotron-70B-Instruct-HF/discussions/22)
25. Discussion on Qwen2-72B - Hugging Face (https://huggingface.co/Qwen/Qwen2-72B/discussions/3)
26. Discussion on Qwen2.5-VL-72B-Instruct - Hugging Face (https://huggingface.co/Qwen/Qwen2.5-VL-72B-Instruct/discussions/3)
27. Best VRAM Cheat Sheet for Local LLMs: Every Model, Every Quant | InsiderLLM - insiderllm.com (https://insiderllm.com/guides/vram-requirements-local-llms/)
28. vLLM vs Ollama vs llama.cpp vs TGI vs TensorRT-LLM: 2025 Guide - ITECS Team (https://itecsonline.com/post/vllm-vs-ollama-vs-llama.cpp-vs-tgi-vs-tensort)
29. Benchmarking NVIDIA TensorRT-LLM - Jan.ai (https://www.jan.ai/post/benchmarking-nvidia-tensorrt-llm)
30. Best GPUs for LLM Inference in 2025 - localllm.in (https://localllm.in/blog/best-gpus-llm-inference-2025)
31. Ollama VRAM Requirements for Local LLMs - localllm.in (https://localllm.in/blog/ollama-vram-requirements-for-local-llms)
32. Mastering LLM Tool Calling: The Complete Framework for Connecting Models to the Real World - MachineLearningMastery.com (https://machinelearningmastery.com/mastering-llm-tool-calling-the-complete-framework-for-connecting-models-to-the-real-world/)
33. GPU Inference Servers Comparison: Triton vs TGI vs vLLM vs Ollama - Nir Adler’s Blog (https://blog.niradler.com/gpu-inference-servers-comparison-triton-vs-tgi-vs-vllm-vs-ollama)

34. Function Calling with LLMs | Prompt Engineering Guide - Prompt Engineering Guide (https://www.promptingguide.ai/applications/function_calling)
35. Function Calling - Qwen - Qwen Docs (https://qwen.readthedocs.io/en/latest/framework/function_call.html)
36. r/AI_Agents on Reddit: Function Calling in LLMs – Real Use Cases and Value? - Reddit (https://www.reddit.com/r/AI_Agents/comments/1iio39z/function_calling_in_llms_real_use_cases_and_value/)
37. r/LocalLLaMA on Reddit: Llama3-70B performs very similarly to Mistral - Reddit (https://www.reddit.com/r/LocalLLaMA/comments/1c89hj3/llama370b_performs_very_similarly_to_mistral/)
38. r/LocalLLaMA on Reddit: New RAG benchmark including Llama3 70B and 8B - Reddit (https://www.reddit.com/r/LocalLLaMA/comments/1c9whsv/new_rag_benchmark_including_llama3_70b_and_8b/)
39. r/LocalLLaMA on Reddit: Llama 3 70B Instruct works surprisingly well on... - Reddit (https://www.reddit.com/r/LocalLLaMA/comments/1cj4det/llama_3_70b_instruct_works_surprisingly_well_on/)
40. r/LocalLLaMA on Reddit: Are there any good newer models similar to... - Reddit (https://www.reddit.com/r/LocalLLaMA/comments/1ecnupa/are_there_any_good_newer_models_similar_to/)
41. r/LocalLLaMA on Reddit: Hardware requirements to run Llama 3 70B on a... - Reddit (https://www.reddit.com/r/LocalLLaMA/comments/1eiwnqe/hardware_requirements_to_run_llama_3_70b_on_a/)
42. r/comfyui on Reddit: Please explain the use of VRAM and some models... - Reddit (https://www.reddit.com/r/comfyui/comments/1nk6pso/please_explain_the_use_of_vram_and_some_models/)
43. r/StableDiffusion on Reddit: Manually setting VRAM limits in Comfy for... - Reddit (https://www.reddit.com/r/StableDiffusion/comments/1ewq140/manually_setting_vram_limits_in_comfy_for/)
44. r/StableDiffusion on Reddit: Are dual GPUs out of the question for local AI... - Reddit (https://www.reddit.com/r/StableDiffusion/comments/1i8y03g/are_dual_gpus_out_of_the_question_for_local_ai/)
45. ComfyUI-MultiGPU - ComfyUI Nodes - RunComfy (https://www.runcomfy.com/comfyui-nodes/ComfyUI-MultiGPU)
46. Benchmarking Local Models: Minimax25 vs Llama 3 vs Mistral - SitePoint (https://www.sitepoint.com/benchmarking-local-models-minimax25-vs-llama-3-vs-mistral/)
47. Local LLM Hardware Requirements: Mac vs PC 2026 | SitePoint - SitePoint (https://www.sitepoint.com/local-llm-hardware-requirements-mac-vs-pc-2026/)
48. Ollama vs vLLM: When to Scale Your Local AI Stack - SitePoint (https://www.sitepoint.com/ollama-vs-vllm-scaling-local-ai-stack)
49. Nvidia’s RTX 3090 is a surprisingly capable AI workhorse - The Register (https://www.theregister.com/2024/08/23/3090_ai_benchmark/)
50. Running private LARGE AI Models on Your Hardware: A Guide to Scaling Local LLMs - Weisser Zwerg (https://weisser-zwerg.dev/posts/local-llm-large-llms/)